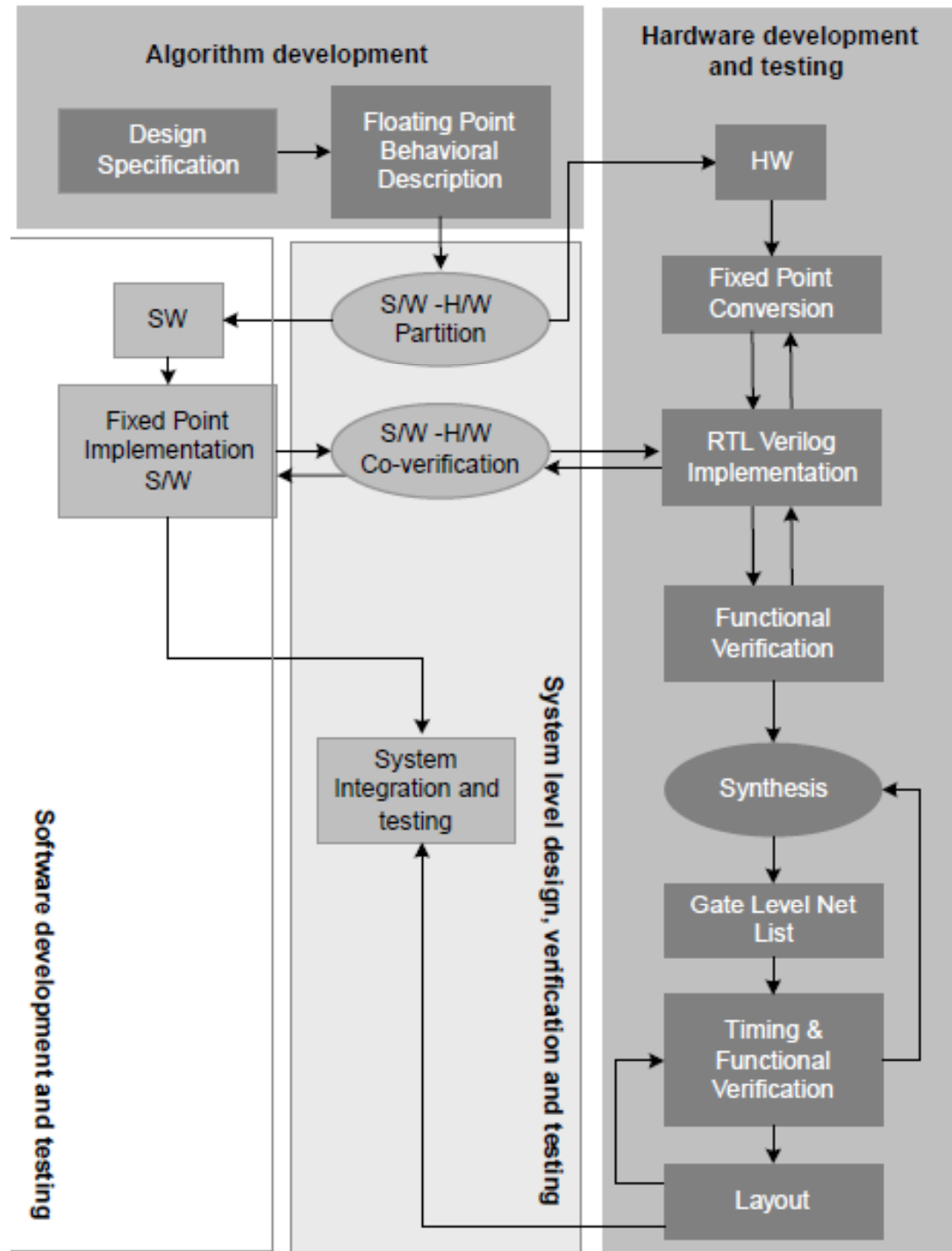




Using a Hardware Description Language

Design Flow-

A bigger picture



Logic Synthesis

- The code written in RTL Verilog is synthesized for gate level implementation.
- The synthesis process takes the RTL Verilog and translates it into an optimized gate level netlist.
- User specifies design constraints and the target technology in the form of a standard cell library.
- The library has standard basic logic gates such as AND and OR, or macro cells like adders, multipliers, flip flops, multiplexers and so on.
- The tool completely converts the design described in RTL hardware description language into a design that contains standard cells.

Verilog

- Design steps
 - Modeling, Simulation and Synthesis
- Verilog is used for modeling design and writing stimulus for testing and simulation on simulation tools like Modelsim®
- Verilog has two sets of constructs, stimulus and synthesis
- Stimulus constructs are used for writing stimulus
- Synthesis constructs are used to write actual design
- Some language constructs and their use in a Verilog description make simulation efficient and they are ignored by synthesis tools

Verilog Standards

- 1995: IEEE Standard 1364-1995 (Verilog 95)
- 2002: IEEE Standard 1364-2001 (Verilog 2001)
- 2003: IEEE Standard 1364-2001 revision C
- 2005: IEEE Standard 1364-2005 (Verilog 2005)
- 2005: IEEE Standard 1800-2005 (SystemVerilog)
- Verilog 2001 is recommended for use
- SystemVerilog support full 'C/C++' language just for stimulus, synthesis is same as previous

Using the Verilog HDL

- The Basic Building Block: Module
- Modules are:
 - Declared
 - Instantiated
- Modules declarations cannot be nested
- Modules can be interconnected to describe the structure of your digital system
- Modules start with keyword module and end with keyword endmodule
- Modules have ports for interconnection with other modules
- By default output is wire if not mentioned as reg

Module- Example

- Module rca(input a,
 - Input b,
 - Input cin,
 - Output wire [3:0] s,
 - Output cout);
 - Assign s[3]= a+b+cin;
- endmodule

Two styles of module templates

```
module FA(a, b, c_in, sum,  
c_out);
```

```
input a, b, c;
```

```
ouput sum, c_out;
```

```
assign {c_out, sum} = a+b+c_in;
```

```
endmodule
```

(a)

```
module FA(  
input a, b, c_in,
```

```
output sum, c_out);
```

```
assign {c_out, sum} = a+b+c_in;
```

```
endmodule
```

```
endmodule
```

(b)

Hierarchical Design

- Verilog code contains a top-level module and zero or more instantiated modules
- The top-level module is not instantiated anywhere
- Several copies of a lower-level module may exist
- Each copy stores its own values of regs/wires
- Ports are used for interconnections to instantiated Modules
- Order of ports in module definition determine order for connections of instantiated module

Example

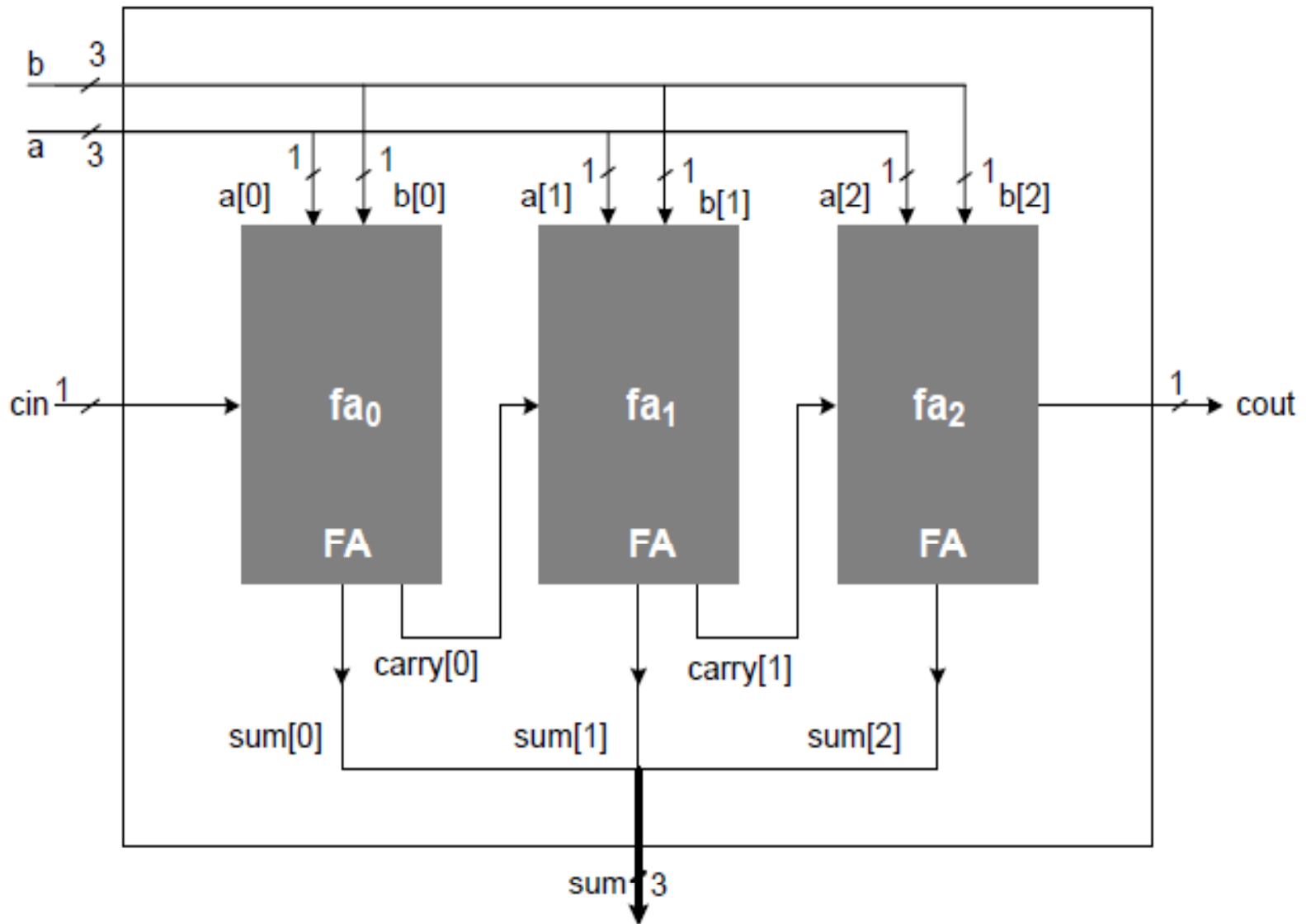


Figure 2.5 Design of a 3 bit RCA using instantiation of three FAs

Example

```
module RCA(  
    input [2:0] a, b,  
    input c_in,  
    output [2:0] sum,  
    output c_out);  
  
    wire carry[1:0];  
  
    // module instantiation  
    FA fa0(a[0], b[0], c_in,  
        sum[0], carry[0]);  
    FA fa1(a[1], b[1], carry[0],  
        sum[1], carry[1]);  
    FA fa2(a[2], b[2], carry[1],  
        sum[2], c_out);  
  
endmodule
```

```
module RCA(  
    input [2:0] a, b,  
    input c_in,  
    output [2:0] sum,  
    output c_out);  
  
    wire carry[1:0];  
  
    // module instantiation  
    FA fa0(.a(a[0]), .b(b[0]),  
        .c_in(c_in),  
        .sum(sum[0]),  
        .c_out(carry[0]));  
    FA fa1(.a(a[1]), .b(b[1]),  
        .c_in(carry[0]),  
        .sum(sum[1]),  
        .c_out(carry[1]));  
    FA fa2(.a(a[2]), .b(b[2]),  
        .c_in(carry[1]),  
        .sum(sum[2]),  
        .c_out(c_out));  
  
endmodule
```

Logic Values

Table 2.1 Possible values a bit may take in Verilog

0	Zero, logic low, false, or ground
1	One, logic high, or power
x	Unknown
z	High impedance, unconnected, or tri state port

- Colors in simulator?

Data Types

- **Nets**

- Nets are physical connections between components
- The net data types are: wire, tri, wor, trior, wand, triand, tri0, tri1, supply0, supply1 and trireg
- Verilog code mostly uses the wire data type
- A variable of type wire represents one or multiple bit values. E.g. x[3:0]
- It can be assigned a value in an expression only once

Data Types

- **Registers**
 - Unless variable of this type is modified it retains previously assigned value
 - Register type is denoted by reg
 - Does not necessarily imply a hardware register
 - Can be used to build memories, flip flops etc.

Variable declaration

- Wire
 - wire [<signed>] [<range>] <net_name>;
 - Wire signed [7:0] x;
- Register
 - reg [<signed>] [<range>] <reg_name> ;
 - Reg [7:0] y;
- Memory
- reg [<range>] <memory_name> [<start_addr> :<end_addr>];

Constants

- Decimal (default)
 - 13, 'd13, 8'd13
- Binary
 - 4'b1101
- Octal
 - 4'o15
- Hexadecimal
 - 4'hd, 16'h34f8

Levels of Abstractions

- Switch Level
 - The lowest level of abstraction is the switch or transistor Level Modeling (obsolete)
- Gate Level
 - Synthesis tools compile high level code and generate code at gate level
- Dataflow Level
 - The level of abstraction higher than the gate level
- Behavioral Level
 - In more complex digital designs, priority is given to the performance and behavior at algorithmic level

Gate Level

- Designs are build from gate primitives
- Verilog has built-in gate-level primitives NAND, NOR, AND, OR, XOR, BUF, NOT, and some others
- Describe the circuit using logic gates as you have see in an implementation of a circuit in basic logic design course
- The delay can also be modeled
- Typical gate instantiation is
 - AND #delay instance-name (out, in1, in2, in3,...)

Example

```
Module (input x1, x2, x3,  
output f);  
Wire g, k, h;  
And g1(g,x1,x2);  
Not g2(k,x2);  
And g3(h, k, x3);  
Or g4(f, g, h);
```

```
Endmodule
```

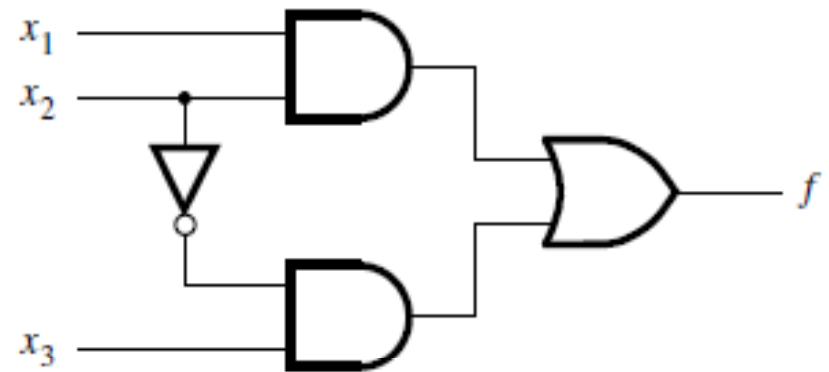


Figure 2.30 A simple logic function.

Dataflow Modeling

- Expressions, operands and operators form the basis of dataflow modeling.
- Expression starts with the keyword assign
- Example:

Module.....

Assign $f = (x_1 \& x_2) \mid (\sim x_2 \& x_3);$

$F(k) = x(k) + k_0 * F(k-1)$

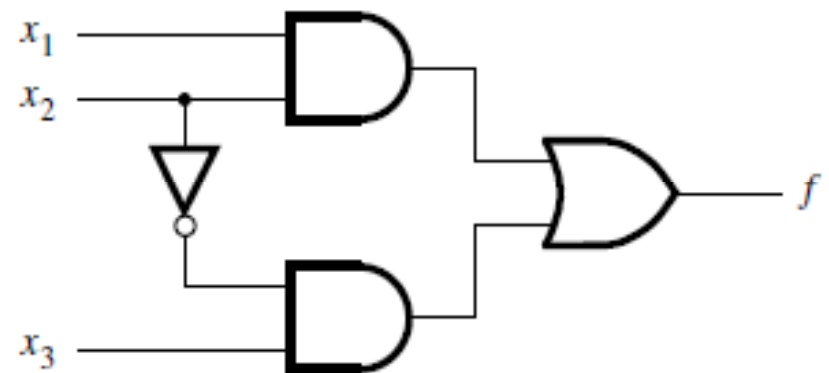


Figure 2.30 A simple logic function.

Behavioral modeling

- Models the behavior of the circuit
- Constructs
 - If-else, case

```
if (x2 == 1)
    f = x1;
else
    f = x3;
```

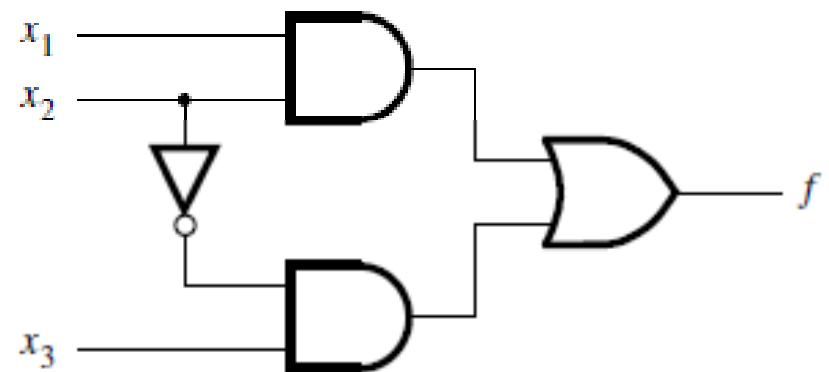


Figure 2.30 A simple logic function.

Dataflow Modeling

Arithmetic operators

Table 2.4 Arithmetic operators

Operator type	Operator symbol	Operation performed
Arithmetic	*	Multiply
	/	Divide
	+	Add
	-	Subtract
	%	Modulus
	**	Power

- Circuits?

Example

```
Assign c = a-b;  
Wire [8:0] y, [7:0] a,b, [15:0] x;  
Assign y = a+b;  
Assign x = a*b;  
Assign f = a/b; only for simulation  
MULT18x18 m1(t, a, b);
```

Conditional operator

Table 2.5 Conditional operator

Operator type	Operator symbol	Operation performed
Conditional	?:	Conditional

- Example

`X = sel? a:b;`

`If(sel)`

`x=a;`

`Else`

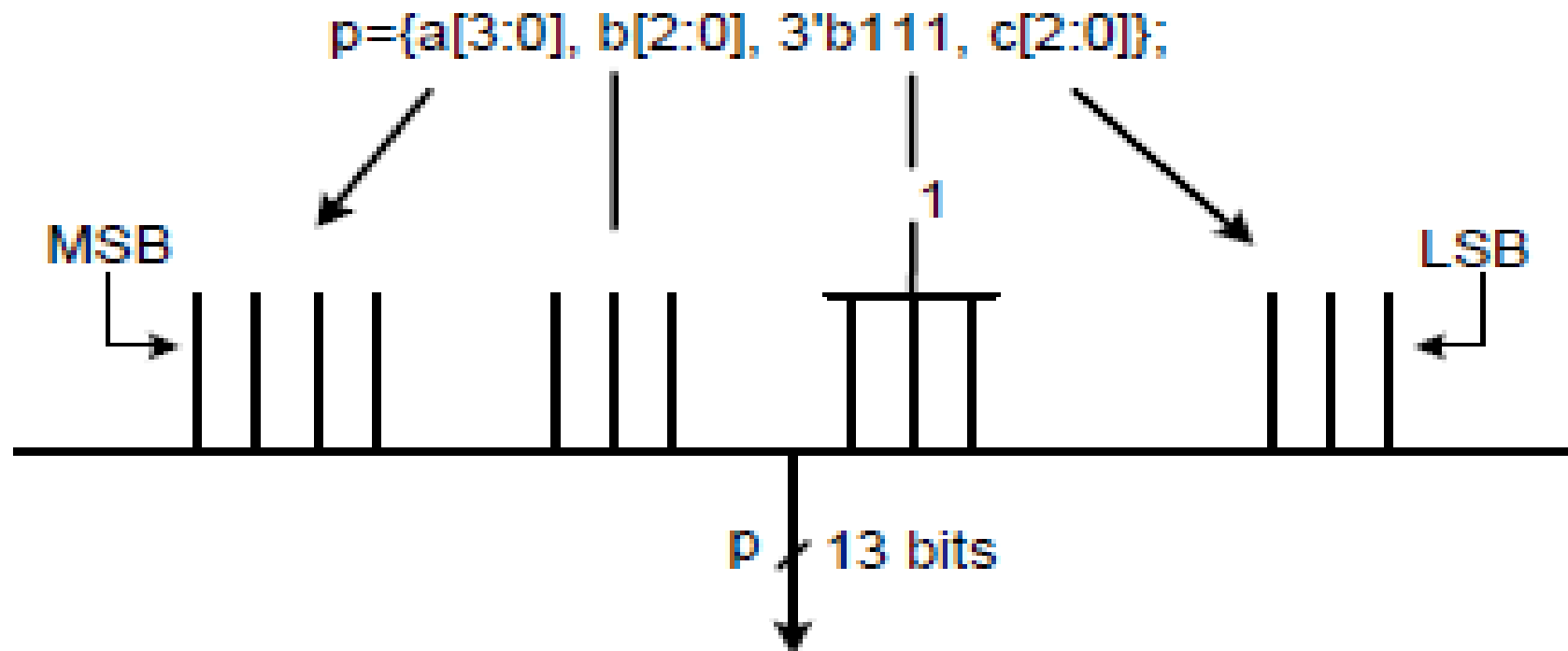
`x=b;`

Concatenation & Replication operators

Table 2.6 Concatenation and replication operators

Operator type	Operator symbol	Operation performed
Concatenation	{ }	Concatenation
Replication	{ { } }	Replication

Concatenation



$\{\text{cout}, y[7:0]\} = a[7:0] + b[7:0]$

Replication

- $A = 2'b01$;
- $B = \{4\{A\}\}$ // the replication operator
- Thus $B = ?$ **$8'b01010101$**

Shift operators

Table 2.8 Shift operators

Operator type	Operator symbol	Operation performed
Logic shift	$\gg$	Unsigned right shift
	$\ll$	Unsigned left shift
Arithmetic shift	$\ggg$	Signed right shift
	$\lll$	Signed left shift

Example

- Logical shift
 - $A = 6'b101111$;
 - $B = A \gg 2$;
 - $B = ?$ **$6'b001011$**
- Circuit?, Barrel shifter ?
- Arithmetic shift
 - $A = 6'b101111$;
 - $B = A \ggg 2$;
 - $B = ?$ **111011**
 - **$A = 6'b010111$**
 - **$B = 000101$**

Logical & Relational

Table 2.7 Logical operators

Operator type	Operator symbol	Operation performed
Logical	!	Logical negation
		Logical OR
	&&	Logical AND

Table 2.9 Relational operator

Relational operator	Operator symbol	Operation performed
	>	Greater than
	<	Less than
	>=	Greater than or equal to
	<=	Less than or equal to

Example

```
wire[7:0] a,b;  
wire x;  
assign x = (a>18) || (b<24);
```

Circuit?

Reduction operators

Table 2.10 Reduction operators

Operator type	Operator symbol	Operation performed
Reduction	&	Reduction AND
	~&	Reduction NAND
		Reduction OR
	~	Reduction NOR
	^	Reduction XOR
	^~ or ~^	Reduction XNOR

- Example

```
A=4'b1011;  
assign B=&A;
```

```
B= A[0] & A[1] & A[2] & A[3];
```

Bitwise arithmetic

Table 2.11 Bitwise arithmetic operators

Operator type	Operator symbol	Operation performed
Bitwise	$\sim$	Bitwise negation
	$\&$	Bitwise AND
	$\sim\&$	Bitwise NAND
	$ $	Bitwise OR
	$\sim $	Bitwise NOR
	$\wedge$	Bitwise XOR
	$\wedge\sim$ or $\sim\wedge$	Bitwise XNOR

- Example

A=4'b1011;

B=4'b0011;

assign C=A|B;

soln. C=4'b1011

Equality operators

Table 2.12 Equality operators

Operator type	Operator symbol	Operation performed
Equality	==	Equality
	!=	Inequality
	===	Case equality
	!==	Case Inequality

- Example

```
x= (A==3);
```

```
y=(A!=3);
```

```
z = (A===3'b01x);
```

Behavioral Modeling

Behavioral Modeling

- High level language constructs are used
 - if else, case
 - for loop
 - while etc
- All statements come in a procedural block
- Two types of procedural blocks
 - Always
 - initial
- A subset of constructs are synthesizable and called RTL Verilog

Procedural blocks

- Initial and always blocks
- Multiple statements per block
- Procedural assignments: = (blocking assignment), <=(non-blocking assignment)
- Timing control: @, #
- Initial blocks execute once at $t = 0$
- Always blocks execute continuously at $t = 0$ and repeatedly thereafter
- initial block only for stimulus
- reg data types are used
- multiple procedural blocks execute concurrently
- assignment cannot be done in multiple procedural blocks

Example

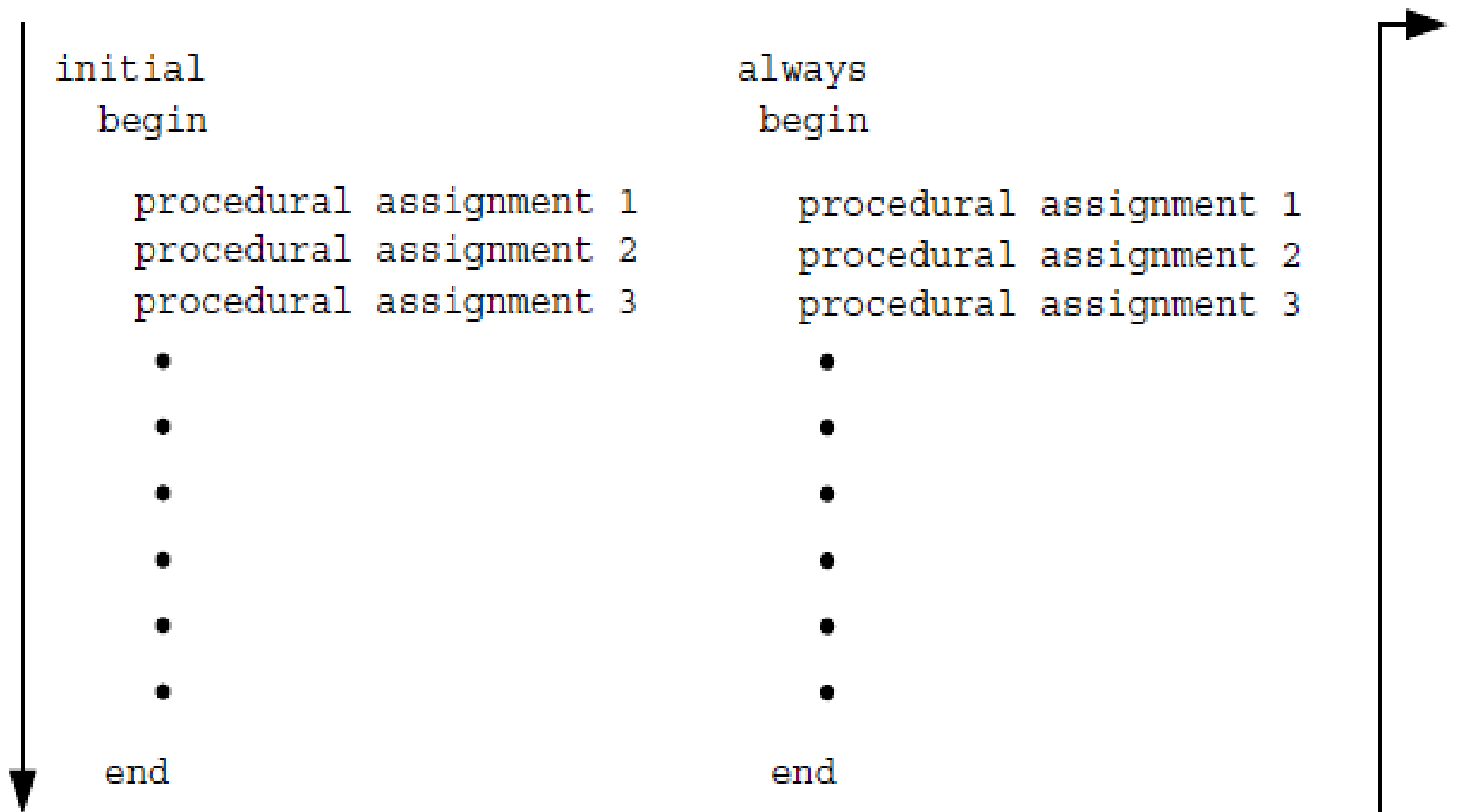


Figure 2.11 Initial and always blocks

Procedural assignments

- Blocking assignments: '='
 - blocks the execution of the subsequent assignments in the sequence
 - are used to model combinational logic
- Non-blocking assignments: '<='
 - these statements execute in parallel
 - are used to model sequential logic

Example

Combination circuit

```
reg sum, carry;  
always @ *  
begin  
    sum = x^y;  
    carry = x&y;  
end
```

Sequential circuit

```
input clk;  
reg sum, carry;  
always @ (posedge clk)  
begin  
    sum = x^y;  
    carry = x&y;  
  
end
```

Time Control # and @

- A built-in variable that represents simulated time, a unit less integer

```
$display ($time, "a %d", a);
```

- This timing control construct '@' is used to model combinational logic as shown here:

```
always@(a or b)  
C = a^b;
```

- The time control @ is used also to model synchronous logic. The code here models a positive edge trigger flip flop:

```
always @ (posedge clk)  
dout <= din;
```

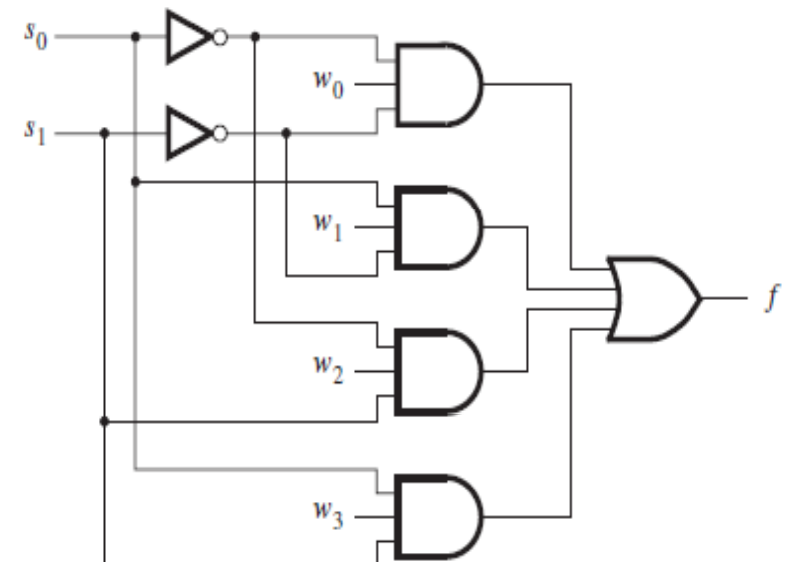
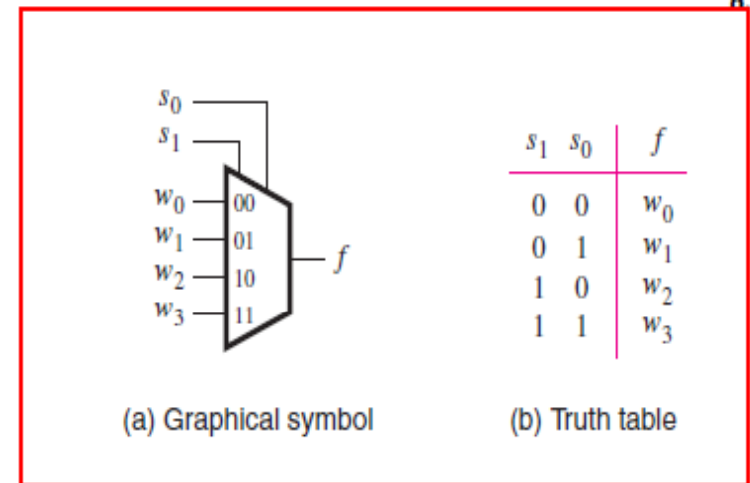
Example- Multiplexer

RTL Verilog Code:

```
always @*  
begin
```

```
    case(sel[1:0])  
        2'b00: f = w0;  
        2'b01: f = w1;  
        2'b10: f = w2;  
        2'b11: f = w3;  
    endcase
```

```
end
```



Decoders

Reg y0, y1, y2, y3;

always@ (posedge clk)

Begin

If (enable)

Begin

case([w0,w1])

0: [y3, y2, y1, y0] = 4'b0001;

1: [y3, y2, y1, y0] = 4'b0010;

2: [y3, y2, y1, y0] = 4'b0100;

3:[y3, y2, y1, y0] = 4'b1000;

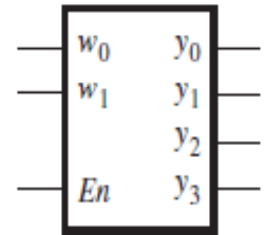
default: [y3, y2, y1, y0] = 4'b0000;

endcase

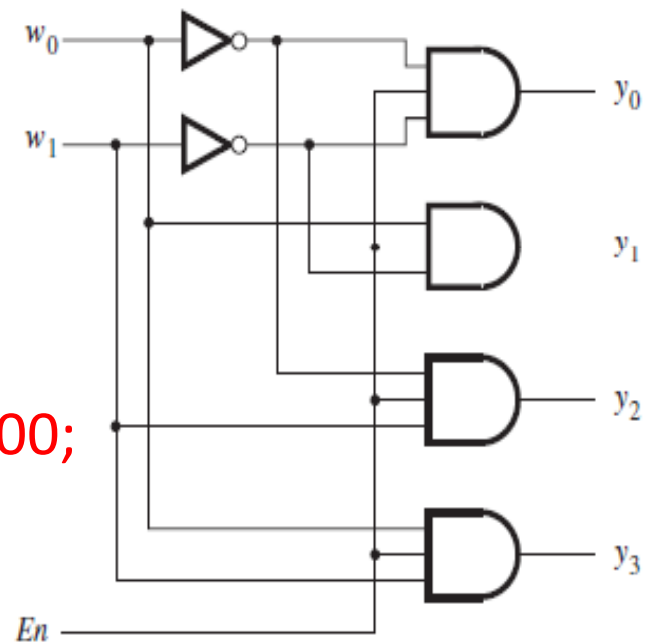
end

<i>En</i>	<i>w</i> ₁	<i>w</i> ₀	<i>y</i> ₀	<i>y</i> ₁	<i>y</i> ₂	<i>y</i> ₃
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

(a) Truth table



(b) Graphical symbol



Decoders

Reg y0, y1, y2, y3;

always@ (posedge clk)

Begin

If (enable)

Begin

if([w0,w1]==0)

[y3, y2, y1, y0] = 4'b0001;

else if([w0,w1]==1)

[y3, y2, y1, y0] = 4'b0010;

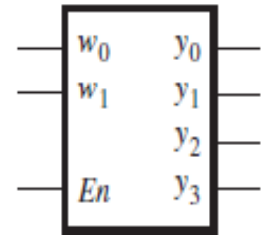
else if

.....

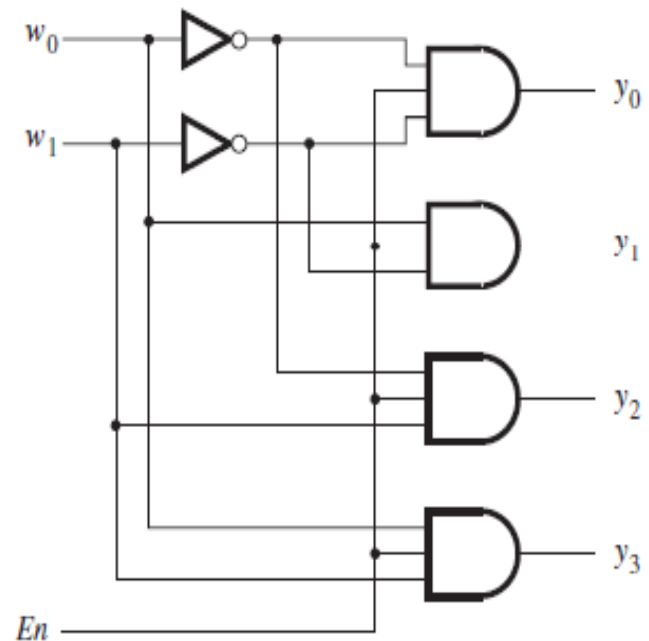
end

<i>En</i>	<i>w</i> ₁	<i>w</i> ₀	<i>y</i> ₀	<i>y</i> ₁	<i>y</i> ₂	<i>y</i> ₃
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1
0	x	x	0	0	0	0

(a) Truth table



(b) Graphical symbol



Encoders

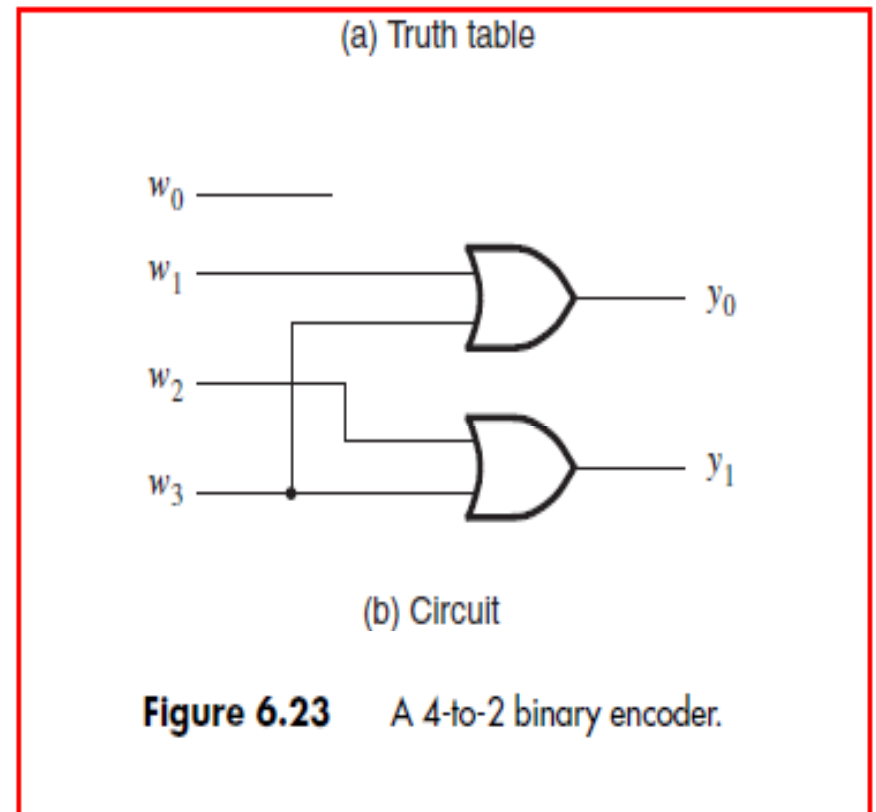
w_3	w_2	w_1	w_0	y_1	y_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Case([w3,w2,w1,w0])

4'b0001: [y1,y0] = 0;

4'b0010:

4'b0100:



Register types

- Simple D-flip flop
- Resettable
 - Asynchronous reset
 - Synchronous reset

Registers/ Flip-flops

Reg q, q_inv;

Input d;

Input clk;

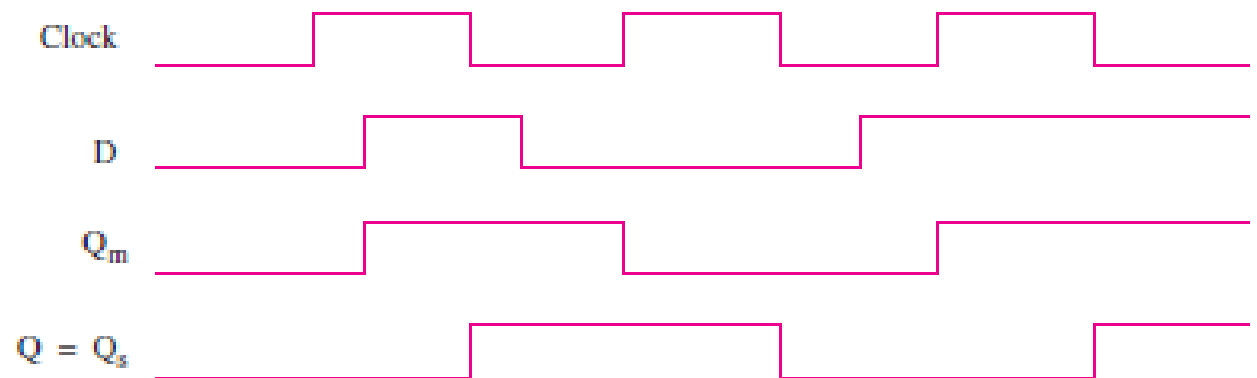
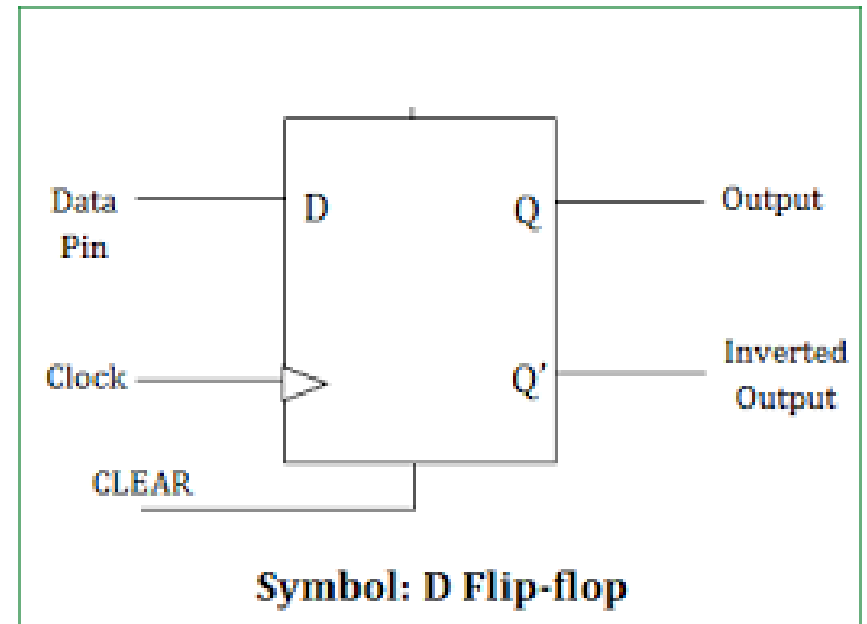
Always @(posedge clk)

Begin

q <= d;

q_inv <= ~d;

end



(b) Timing diagram

Registers/ Flip-flops

```

Reg q, q_inv;
Input d;
Input clk;
Input rst; // asynch rst
Always @(posedge clk or posedge rst)
If(rst)
Begin

```

```

    q <= 0;
    q_inv <= 1;

```

```

end
Begin

```

```

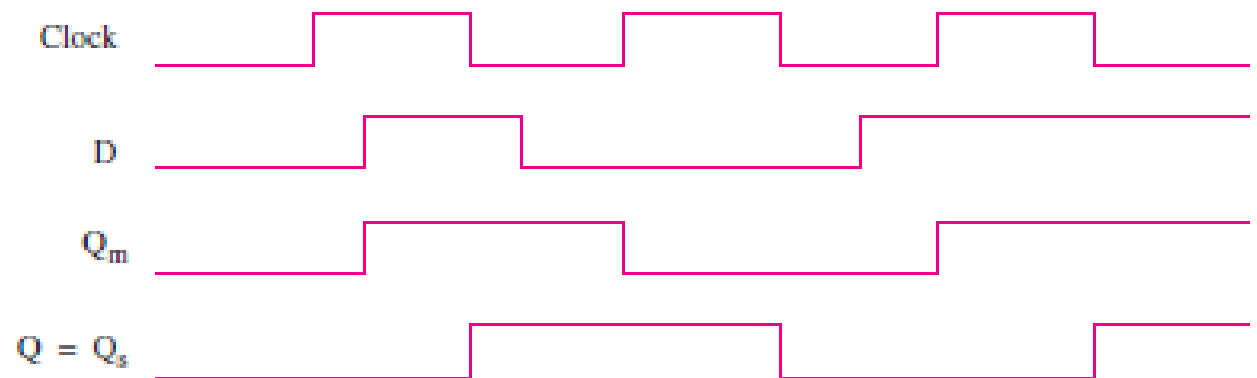
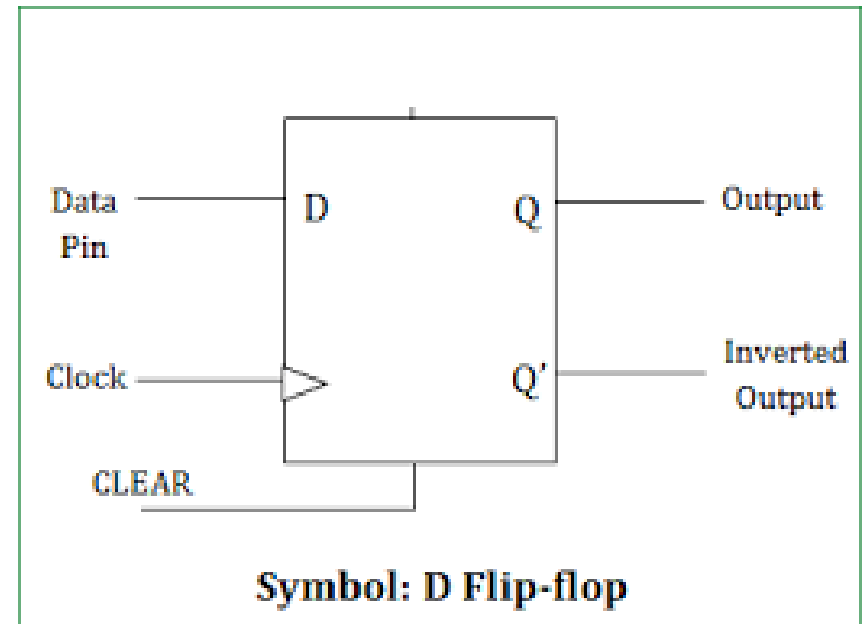
    q <= d;
    q_inv <= ~d;

```

```

end

```



(b) Timing diagram

Registers/ Flip-flops

```

Reg q, q_inv;
Input d;
Input clk;
Input rst; // synch rst
Always @(posedge clk )
If(rst)
Begin

```

```

    q <= 0;
    q_inv <= 1;

```

```

end
Begin

```

```

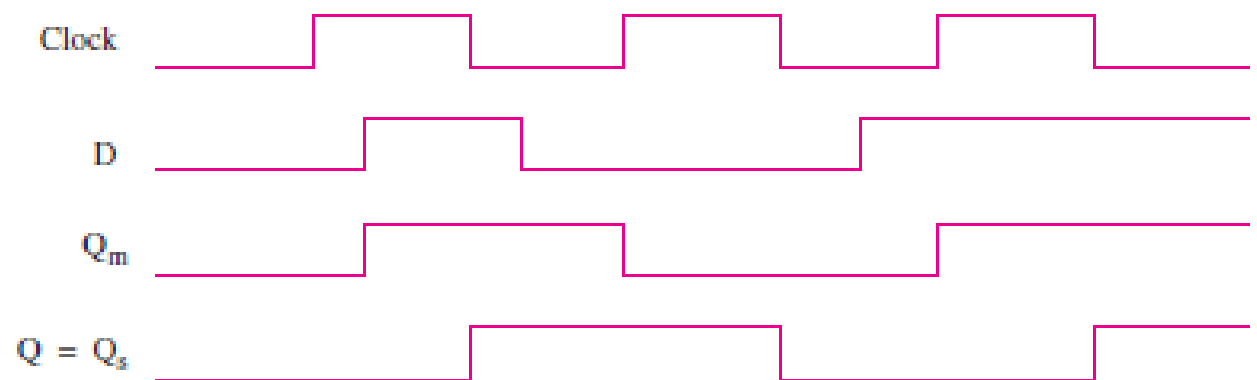
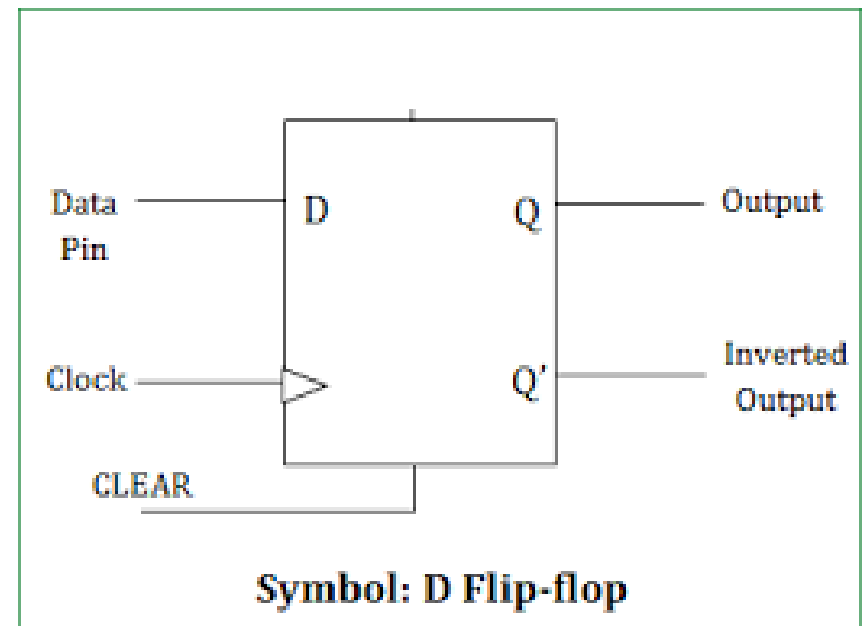
    q <= d;
    q_inv <= ~d;

```

```

end

```



(b) Timing diagram

Accumulator

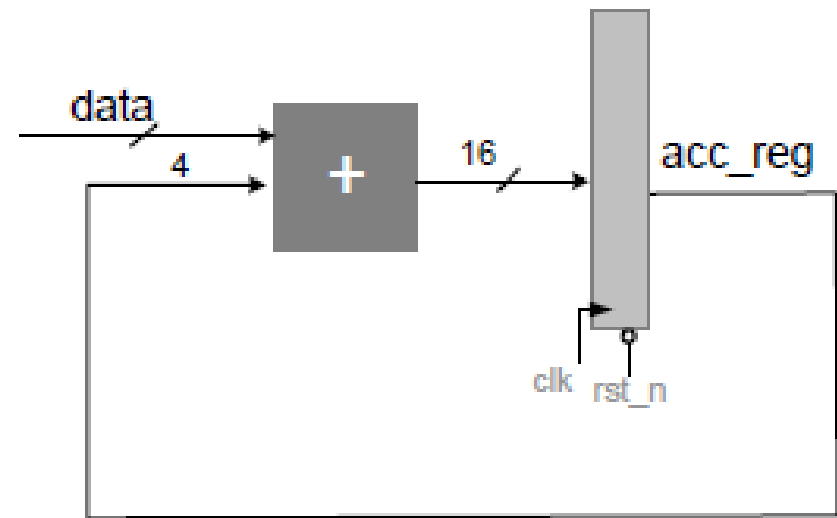
Always @(posedge clk)

If(rst)

acc_reg<= 0;

Else

acc_reg<= data+acc_reg;



Delay pipe-line

$$Y[k] = a_0.x[k] + a_1.x[k-1] + a_2.x[k-2]$$

```
always@(posedge clk)
```

```
Begin
```

```
    D1 <= din;
```

```
    D2<=D1;
```

```
    D3<=D2;
```

```
    Dout<=D3;
```

```
End
```

UP/down counter

```
Reg [3:0] count;  
always@(posedge clk)  
Begin  
If(rst)  
    count <=0;  
Else if(CE)  
    if(LE)  
        count<=load_value;  
    else if(up_down)  
        count <= count +1;  
    else  
        count <= count-1;  
Else  
    count<= count;  
end
```

IIR filter

```
Module iir(input signed [15:0] x,  
Input clk, rst_n,  
Output reg signed [31:0] y);
```

```
Reg signed [31:0] y_reg;
```

```
Always @*  
Y=(y_reg>>>1)+x;
```

```
Always @(posedge clk or negedge rst_n)  
Begin  
If(!rst_n)  
Y_reg<=0;  
Else  
Y_reg<=y;  
End//      y_reg = y(k-1)
```

```
endmodule
```

$$Y(k) = a_0.x(k) + b_0.y(k-1)$$
$$Y(k) = 0.5*y(k-1) + x(k)$$

IIR filter

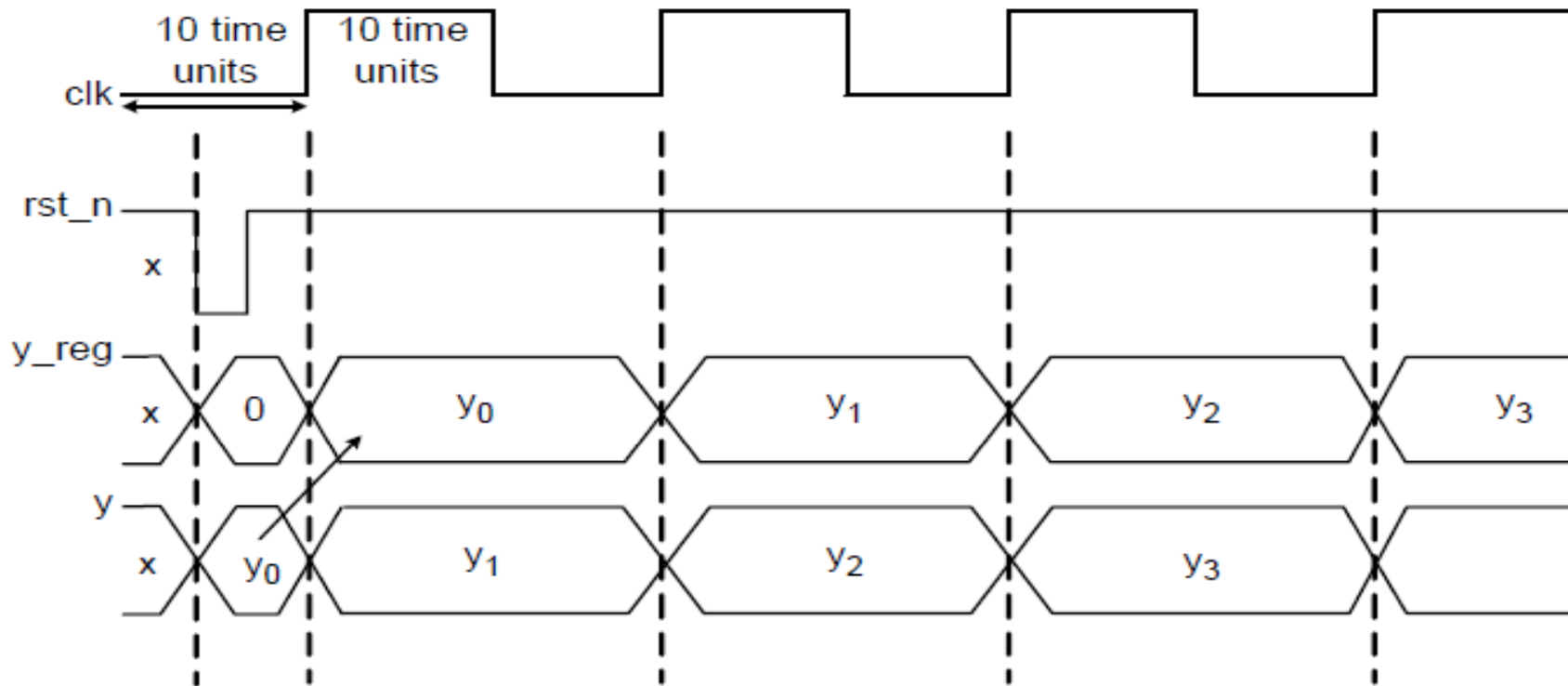


Figure 2.16 Timing diagram for the IIR filter design of example 2.9

RAM

```
parameter RAM_WIDTH = <ram_width>;
parameter RAM_ADDR_BITS = <ram_addr_bits>;

reg [RAM_WIDTH-1:0] <ram_name> [(2**RAM_ADDR_BITS)-1:0];

wire [RAM_WIDTH-1:0] <output_data>;

<reg_or_wire> [RAM_ADDR_BITS-1:0] <address>;
<reg_or_wire> [RAM_WIDTH-1:0] <input_data>;

always @(posedge <clock>)
    if (<write_enable>)
        <ram_name>[<address>] <= <input_data>;

assign <output_data> = <ram_name>[<address>];
```

RAM

```
Parameter RAM_WIDTH = 8;
Parameter RAM_ADDR_BITS = 10;
Reg [RAM_WIDTH-1:0] MEM_1[(2**RAM_ADDR_BITS)-1:0];
Wire [RAM_WIDTH-1:0] dout;
Input [RAM_WIDTH-1:0] din;
Input clk;
Input write_en;
Input [9:0] addr;

Always @(posedge clk)
Begin
    If(write_en)
        MEM_1[addr] <= din;
End
Assign dout = MEM_1[addr];
```

ROM

```
parameter ROM_WIDTH = <ram_width>;

reg [ROM_WIDTH-1:0] <output_data>;
<reg_or_wire> [3:0] <address>;

always @(posedge <clock>)
  if (<enable>)
    case (<address>)
      4'b0000: <output_data> <= <value>;
      4'b0001: <output_data> <= <value>;
      4'b0010: <output_data> <= <value>;
      4'b0011: <output_data> <= <value>;
      4'b0100: <output_data> <= <value>;
      4'b0101: <output_data> <= <value>;
      4'b0110: <output_data> <= <value>;
      4'b0111: <output_data> <= <value>;
      4'b1000: <output_data> <= <value>;
      4'b1001: <output_data> <= <value>;
      4'b1010: <output_data> <= <value>;
      4'b1011: <output_data> <= <value>;
      4'b1100: <output_data> <= <value>;
      4'b1101: <output_data> <= <value>;
      4'b1110: <output_data> <= <value>;
      4'b1111: <output_data> <= <value>;
      default: <output_data> <= <value>;
    endcase
```

0: dout<= 8'h3F;
1: dout<= 8'h43;

Loading Memory Data from a File

- System tasks \$readmemb and \$readmemh are used to load data from a text file written in binary or hexadecimal, respectively, into specified memory.

```
reg [7:0] mem[63:0];  
Reg [0:7] x;  
$readmemb ("memory.dat", mem);
```

Parameters

- There are two main types of parameters, the **parameter** and **local parameter**. A local parameter acts the same as a parameter however its contents cannot be modified via a defparam or a named parameter value assignment in the instantiation.

Example

Design file:

Localparam real pi = 3.14;

Parameter BUS_WIDTH =8;

Reg [BUS_WIDTH-1:0] reg1;

Top_level_module:

My_design #(
.

BUS_WIDTH(16)

)

UUT(
.

A(A),

.B(B)

);

Bi-directional IOs

- Tri-state buffers
 - Only available at FPGA external pins

```
Inout y_io;  
Wire oe, x_in, t_out;  
Assign y_io = oe?t_out:1'bz
```

```
Assign x_in = y_io;
```


Task and for loop

```
Module RCA(  
    Input [3:0] a,b,  
    Input c_in,  
    Output reg c_out,  
    Output reg [3:0] sum);  
    Reg carry[4:0];  
    Integer i;  
    Task FA(  
        Input in1,in2,carry_in,  
        Output reg out, carry_out);  
        {carry_out, out} = in1+in2+carry_in;  
    endtask  
  
    always@*  
    Begin  
        Carry[0]=c_in;  
        For(i=0;i<4;i=i+1)  
            Begin  
                FA(a[i],b[i],carry[i],sum[i],carry[i+1]);  
            End  
        C_out = carry[4];  
    End  
endmodule
```

RTL Coding Guideline

- Avoid Latches in the Design
 - A latch is a storage device that stores a value without the use of a clock.
 - Latches are technology-specific and must be avoided in synchronous designs
 - To avoid latches adhere to coding guidelines
 - Fully specify assignments or use a default assignment
 - For if block there must be an else
 - For case, either check all conditions or use a default

Example

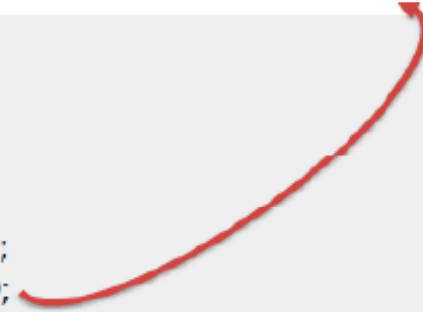
```
input [1:0] sel;
reg [1:0] out_a, out_b;

always @ (*)
begin
    if (sel == 2'b00)
        begin
            out_a = 2'b01;
            out_b = 2'b10;
        end
    else
        out_a = 2'b01;
end
```

■ Use default assignments

```
input [1:0] sel;
reg [1:0] out_a, out_b;

always @ (*)
begin
    out_a = 2'b00;
    out_b = 2'b00;
    if (sel == 2'b00)
        begin
            out_a = 2'b01;
            out_b = 2'b10;
        end
    else
        out_a = 2'b01;
end
```



Example

```
input [1:0] sel;
reg [1:0] out_a, out_b;
always @*
    begin
        out_a = 2'b00;
        out_b = 2'b00;
        if (sel==2'b00)
            begin
                out_a = 2'b01;
                out_b = 2'b10;
            end
        else if (sel == 2'b01)
            out_a = 2'b01;
    end
end
```

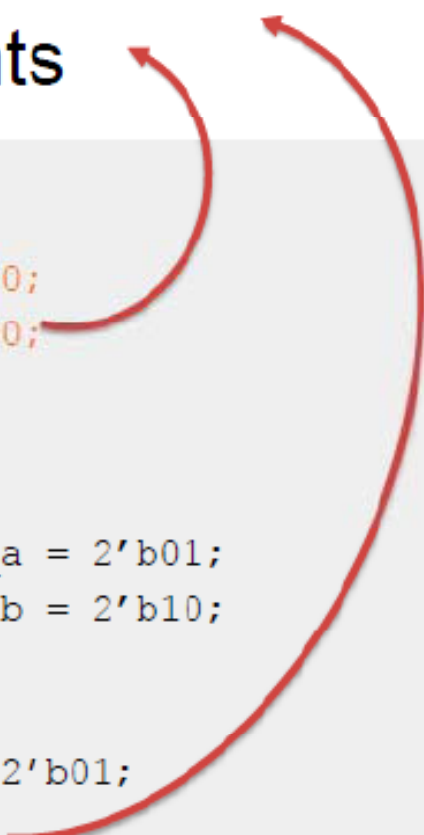
```
always @*
    begin
        out_a = 2'b00;
        out_b = 2'b00;
        if (sel==2'b00)
            begin
                out_a = 2'b01;
                out_b = 2'b10;
            end
        else if (sel == 2'b01)
            out_a = 2'b01;
        else
            out_a = 2'b00;
    end
end
```

Corrected design

Example

Correct use of case statements and default assignments

```
always @*  
begin  
    out_a = 2'b00;  
    out_b = 2'b00;  
    case (sel)  
    2'b00:  
        begin  
            out_a = 2'b01;  
            out_b = 2'b10;  
        end  
    2'b01:  
        out_a = 2'b01;  
    default:  
        out_a = 2'b00;  
    endcase  
end
```



Avoid Combinational Feedback

